

Render pipelines

Y otras cosas :)

Marco Antonio Gómez Martín
Iluminación y Materiales
Grado en Desarrollo de Videojuegos
2025/2026

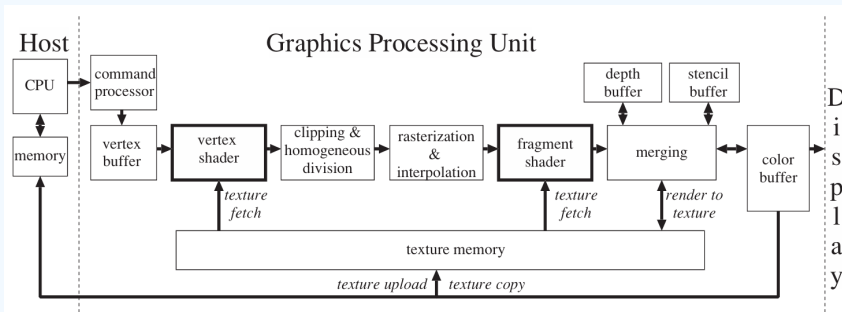
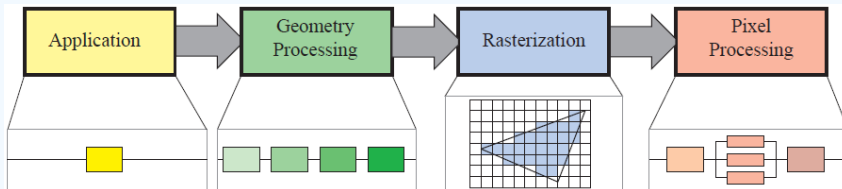


Facultad
de
Informática

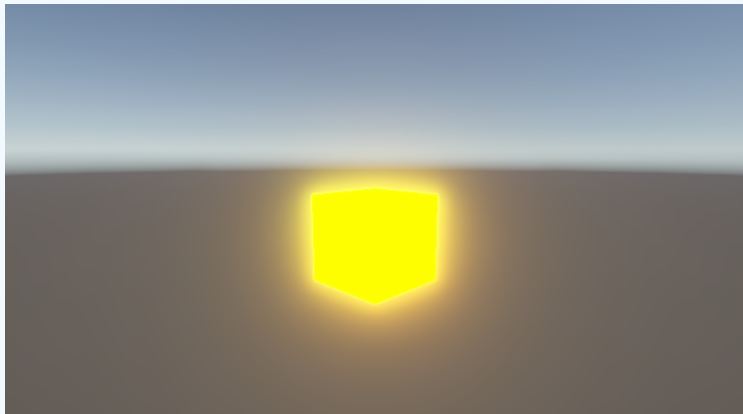


UNIVERSIDAD COMPLUTENSE
MADRID

Pipeline (parcial)



Y esto, ¿cómo se hace?



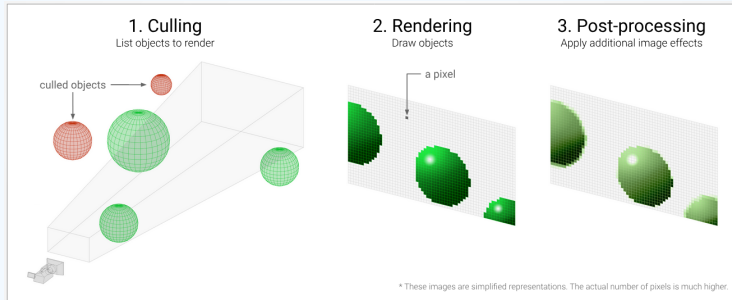
Y esto, ¿cómo se hace?



Y esto, ¿cómo se hace?

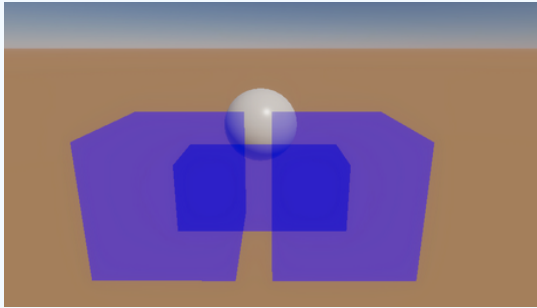


Modelo [muy] simplificado



Orden de pintado

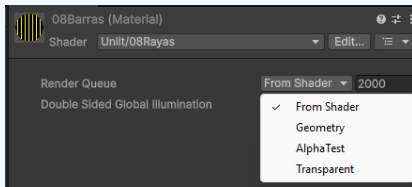
- Primero objetos cercanos: reduce el overdraw.
- Primero objetos lejanos: única alternativa para que funcionen las transparencias.



Orden de pintado

Pintar en dos fases (solución simplificada):

- Objetos opacos (de delante hacia atrás).
- Objetos transparentes (de detrás hacia delante).



```
Tags {  
    "RenderType" = "Opaque"  
    "Queue" = "Geometry"  
    "RenderPipeline" = "UniversalPipeline"  
}
```

Orden de pintado - Unity

Background This render queue is rendered before any others. This value corresponds to a render queue of 1000.

Geometry Geometry is a predefined render queue that Unity uses for opaque geometry.

AlphaTest Alpha tested geometry uses this queue. This value corresponds to a render queue of 2450.

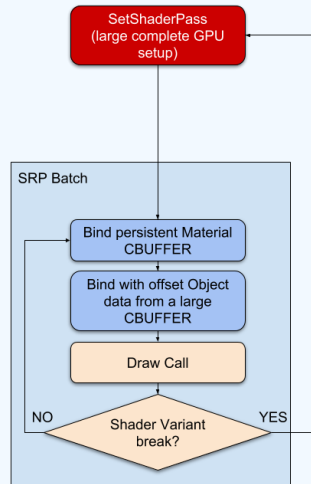
GeometryLast The last render queue that is treated as opaque. This value corresponds to a render queue of 2500.

Transparent This render queue is rendered after Geometry and AlphaTest, in back-to-front order. This value corresponds to a render queue of 3000.

Overlay This render queue is meant for overlay effects, and is rendered last. This value corresponds to a render queue of 4000.

Orden de pintado - Unity

Y todo eso intentando reducir el número de *draw calls* y cambio de contexto en la GPU.



Transparencias simples - Alpha clipping

- Consiste en conseguir *simular* transparencias dejando de pintar algunos píxeles.
- Mucho más eficiente que la transparencia real, pero peor resultado.
- Útil para zonas pequeñas / geometría alejada.

Transparencias simples - Alpha clipping

GTA V (2013)

<https://www.adriancourreges.com/blog/2015/11/02/gta-v-graphics-study/>



Transparencias

- Consistente en *mezclar* el color del pixel ya guardado en el RT con el del material.
- La mezcla es configurable y la hace el *hardware*.
- Si no se configura mezcla (`Blend Off`) el color resultante sobrescribe al anterior y será opaco.
- En Unity se configura en la pasada.

Transparencias

- *source*: color calculado por el PS.
- *destination*: color preexistente en el RT.
- El hardware hace el cálculo.
$$color = (source_factor * source) + (destination_factor * destination)$$
- En el *shader* se especifican los dos factores.
`Blend <source factor> <destination factor>`

Transparencias

One Multiplicar por 1.

Zero Multiplicar por 0 (ignorar valor).

SrcColor Multiplicar por el color del PS.

OneMinusSrcColor Multiplicar por $(1 - \text{color del PS})$.

SrcAlpha Multiplicar por el alpha del PS.

OneMinusSrcAlpha Multiplicar por $(1 - \text{alpha del PS})$.

Src $\rightarrow$ **Dst** Variantes equivalentes sustituyendo *Src* por *Dst*.

SrcAlphaSaturate Multiplica por el mínimo del alpha del PS y el 1-alpha del RT.

Transparencias

Combinaciones habituales

Transparencias

Combinaciones habituales

- `Blend SrcAlpha OneMinusSrcAlpha`

Transparencias

Combinaciones habituales

- Blend SrcAlpha OneMinusSrcAlpha
- Blend One OneMinusSrcAlpha

Transparencias

Combinaciones habituales

- Blend SrcAlpha OneMinusSrcAlpha
- Blend One OneMinusSrcAlpha
- Blend One One

Transparencias

Combinaciones habituales

- Blend SrcAlpha OneMinusSrcAlpha
- Blend One OneMinusSrcAlpha
- Blend One One
- Blend DstColor Zero

Transparencias

Combinaciones habituales

- Blend SrcAlpha OneMinusSrcAlpha
- Blend One OneMinusSrcAlpha
- Blend One One
- Blend DstColor Zero
- Blend Zero SrcColor

Transparencias - Ejemplos

¿Qué mezcla tiene el cubo verde?



Transparencias - Ejemplos

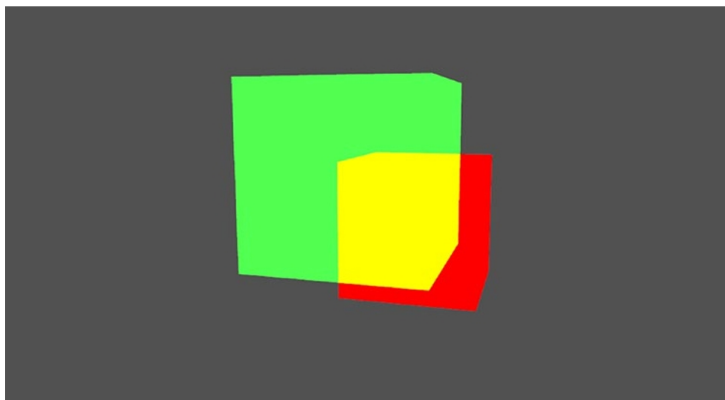
¿Qué mezcla tiene el cubo verde?



Multiplicative blending

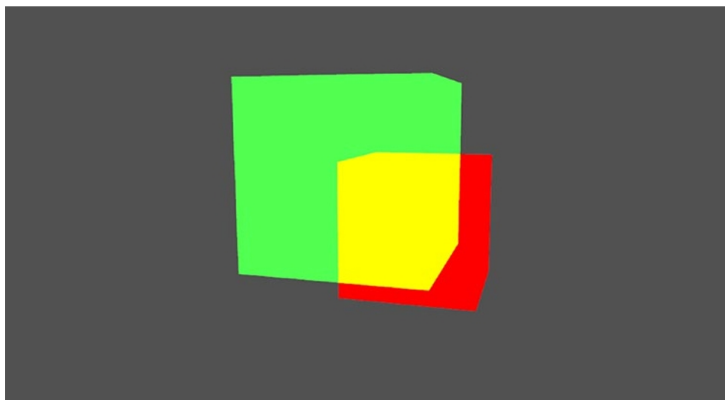
Transparencias - Ejemplos

¿Qué mezcla tiene el cubo verde?



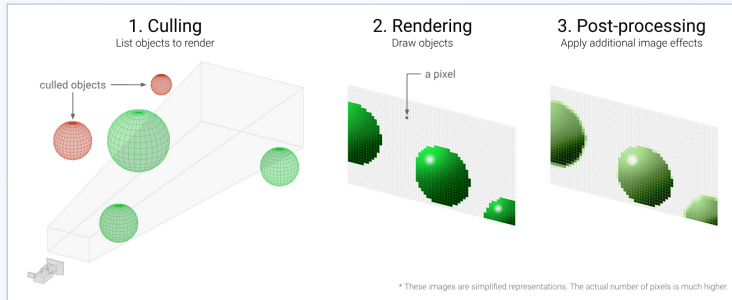
Transparencias - Ejemplos

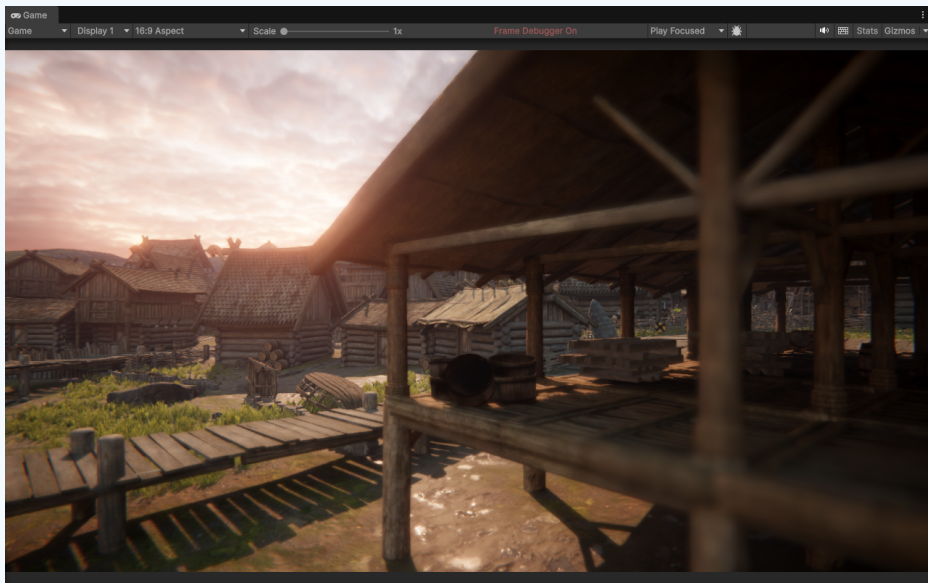
¿Qué mezcla tiene el cubo verde?

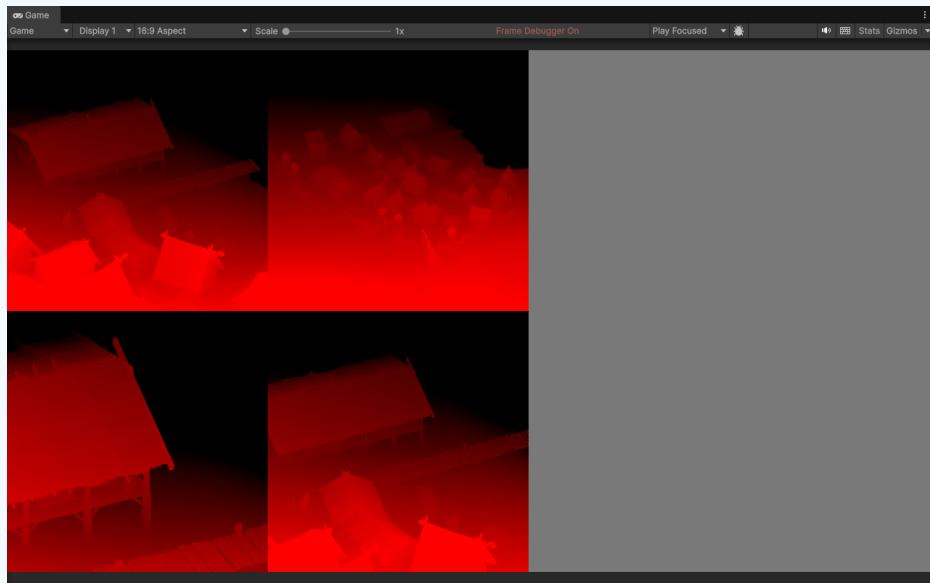


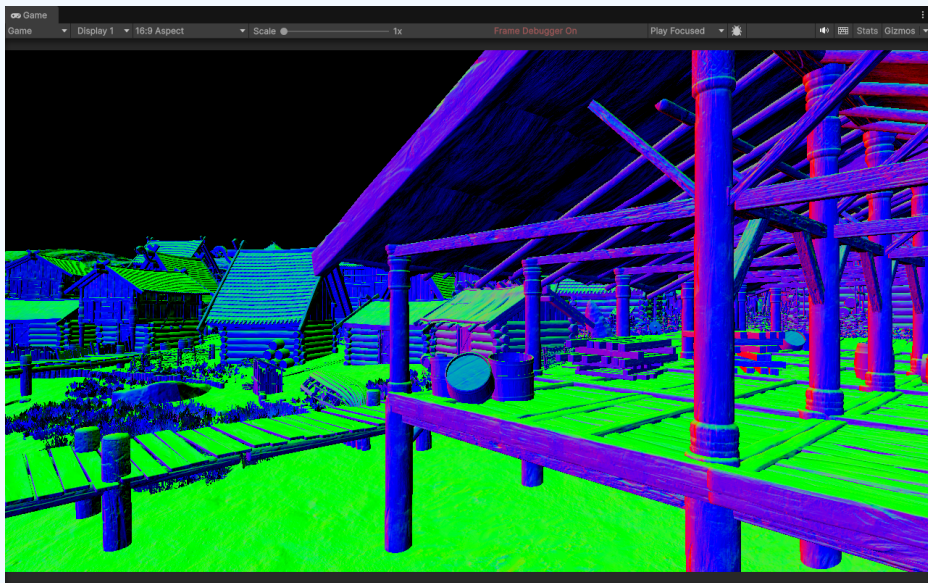
Additive blending

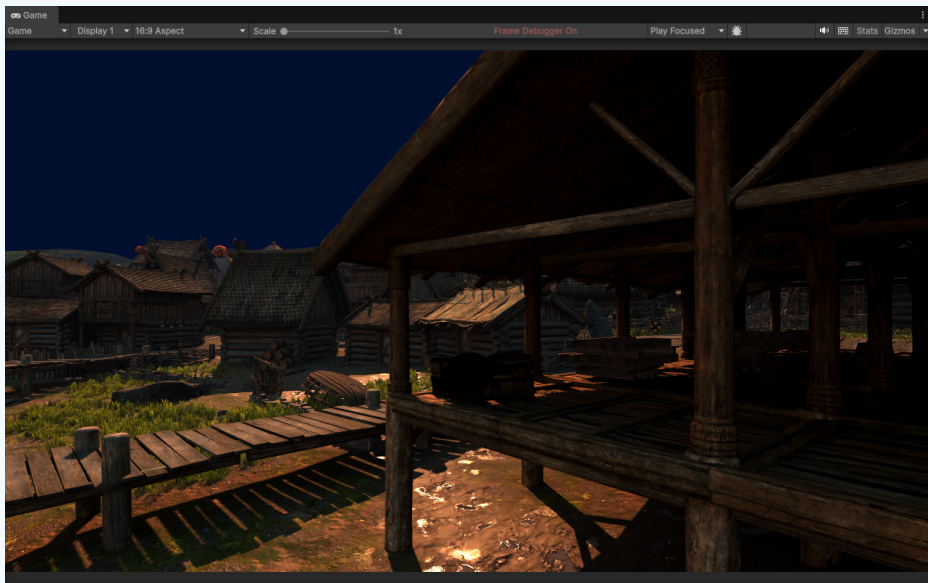
Modelo [muy] simplificado

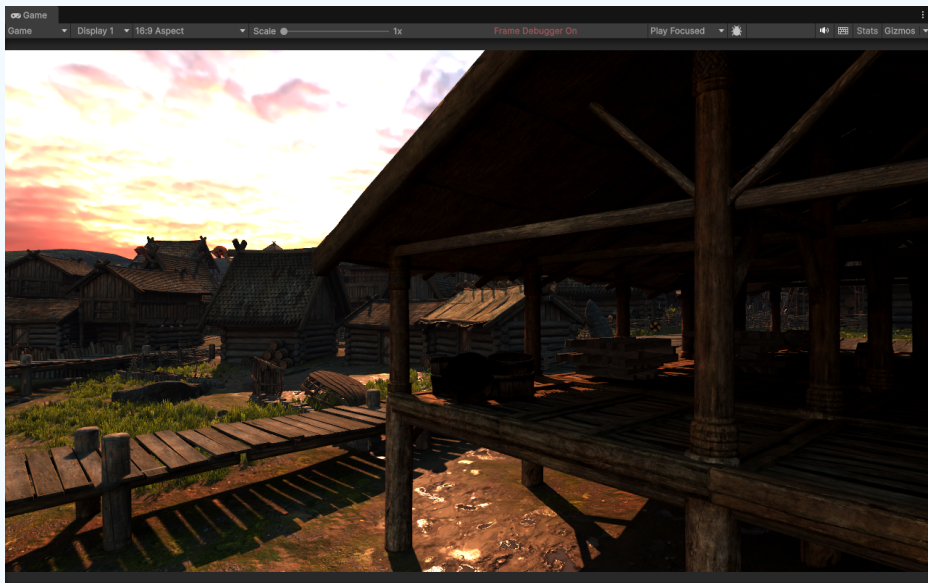


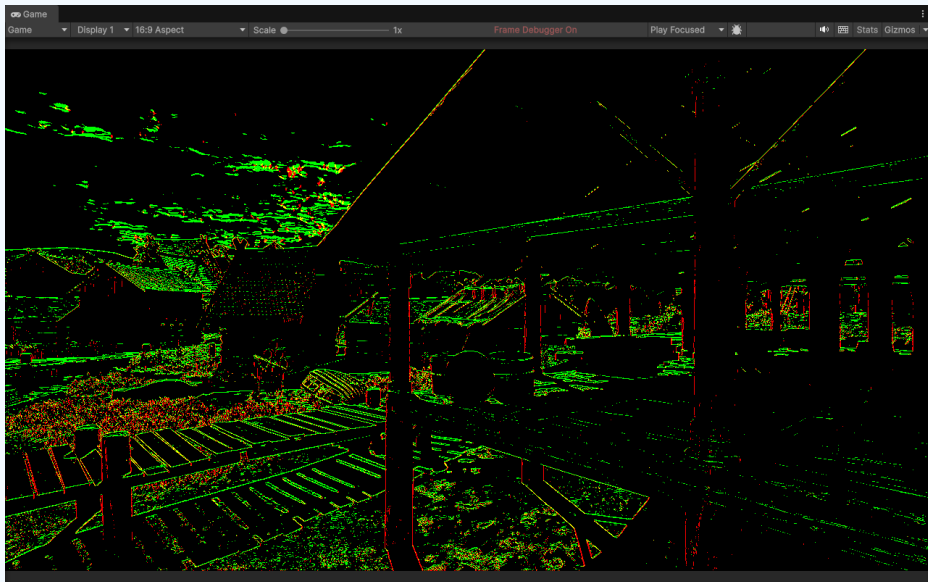


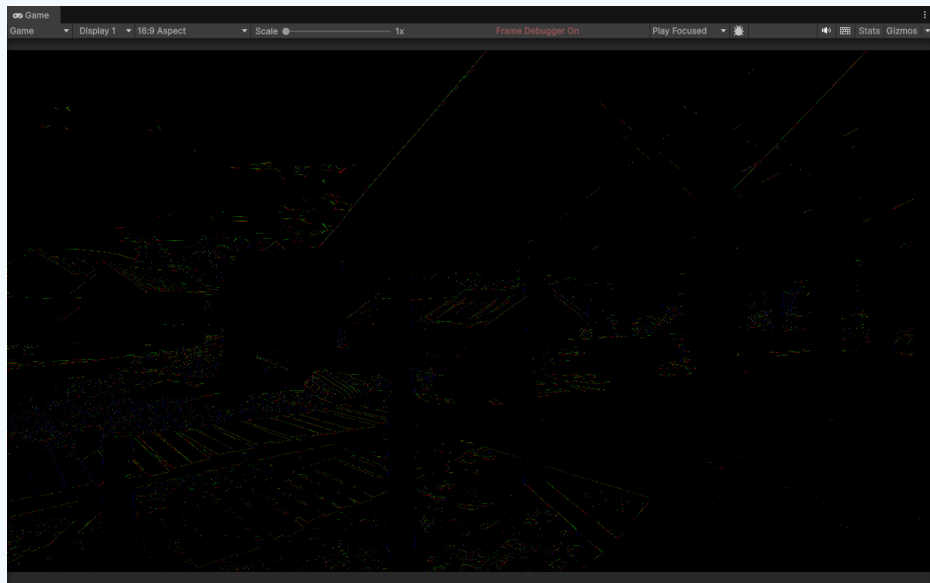


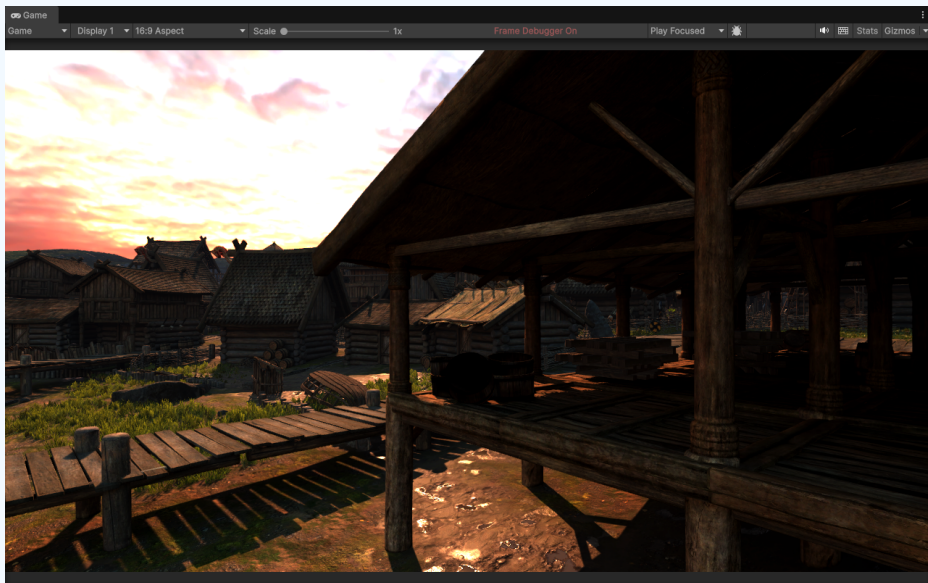


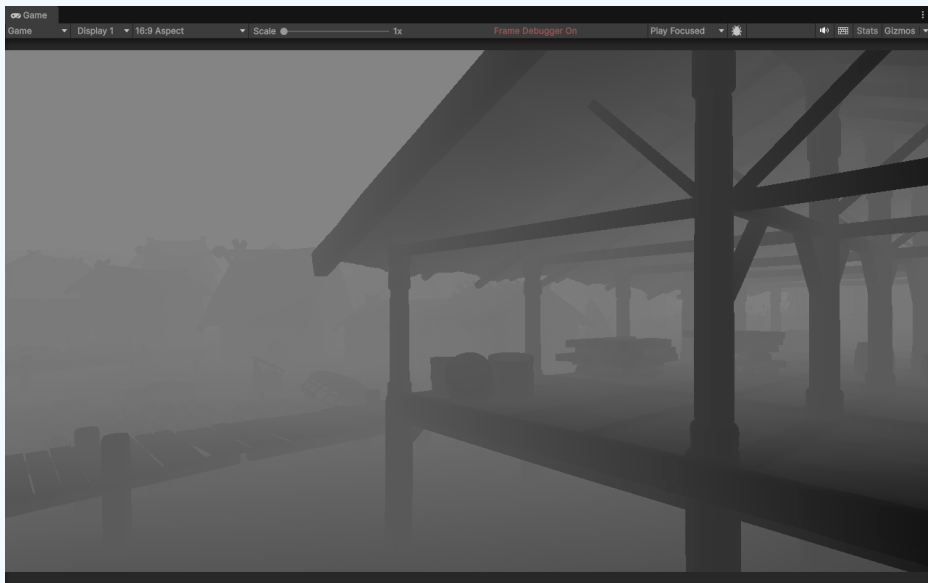


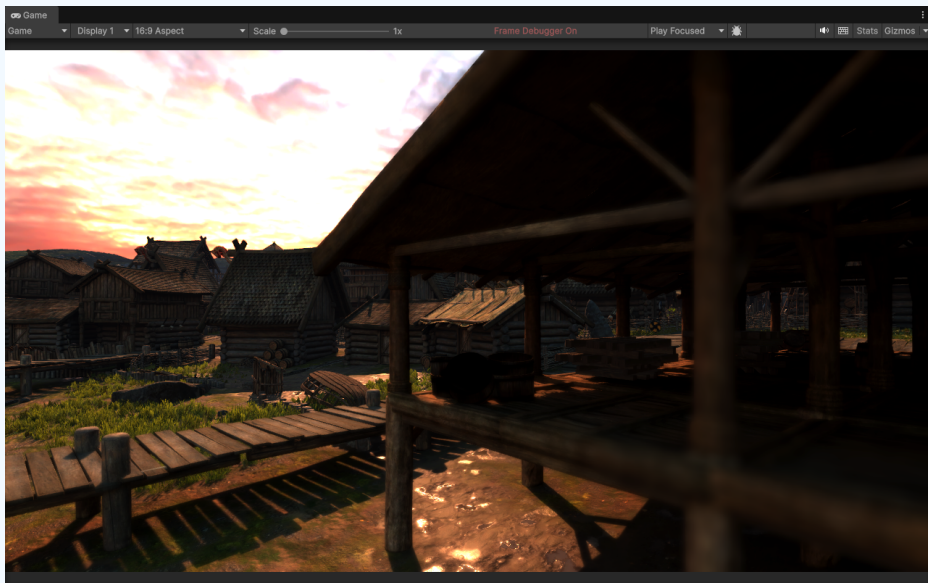


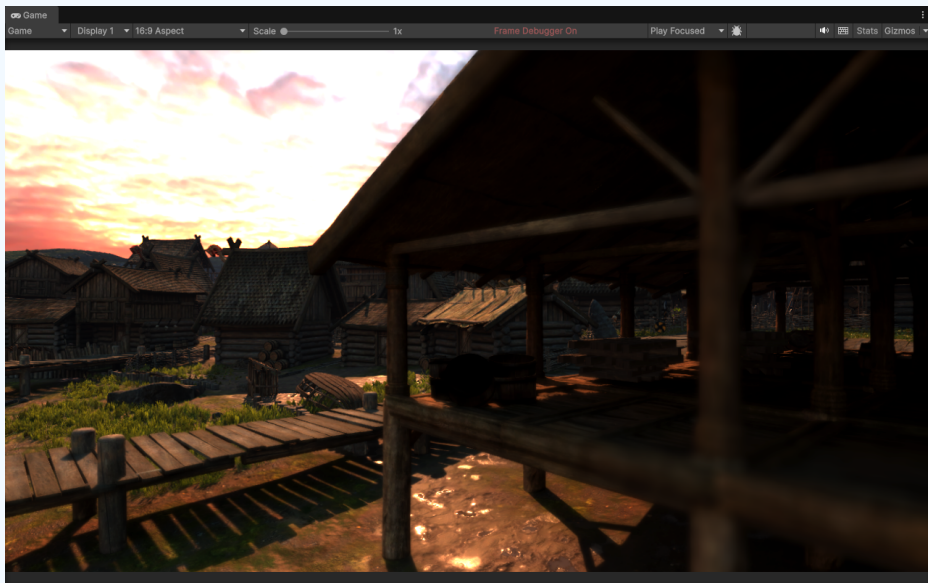


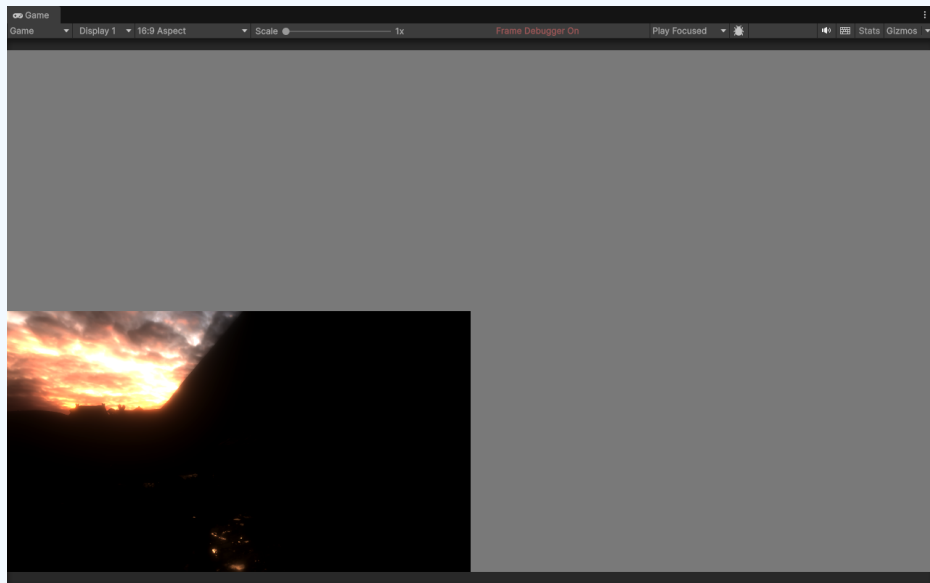


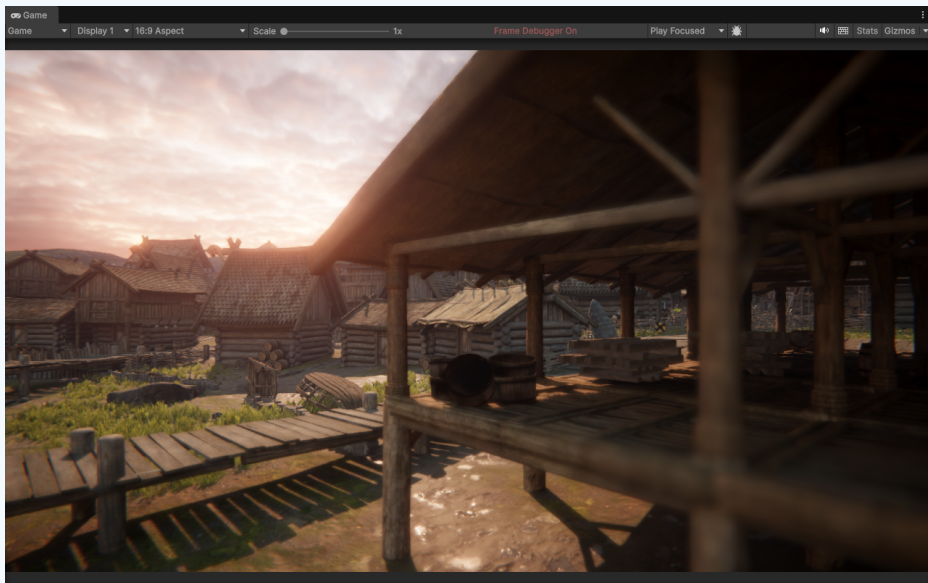




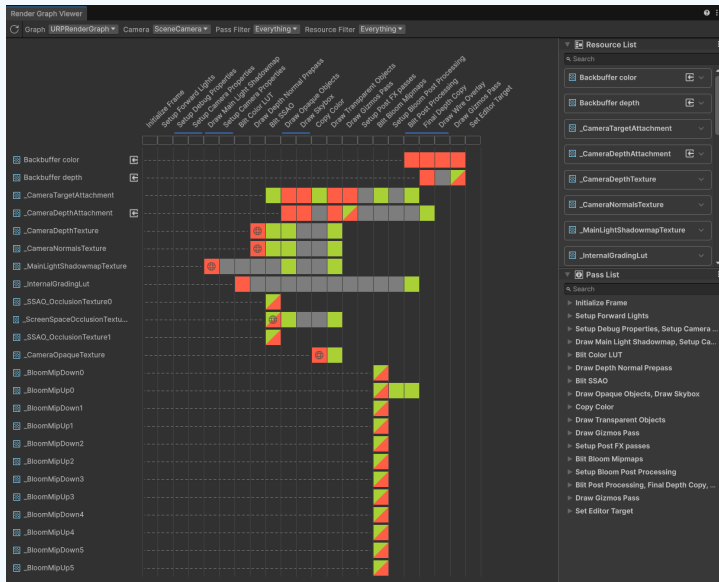




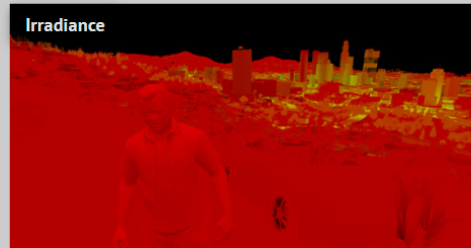
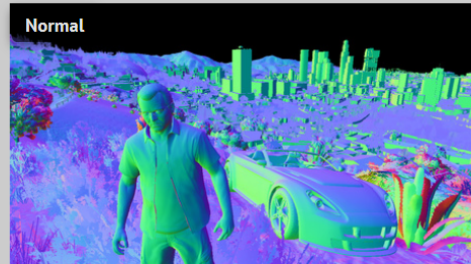




▼ UniversalRenderPipeline.RenderSingleCameraInte	119
▼ ExecuteRenderGraph	119
Clear (depth+stencil)	
▶ (RP 0:0) Draw Main Light Shadowmap	25
Clear (color)	
▶ (RP 1:0) Blit Color LUT	1
Clear (color+depth+stencil)	
▶ (RP 2:0) DrawDepthNormalPrepass	18
Clear (color)	
▶ SSAO	5
Clear (depth+stencil)	
▶ (RP 3:0) DrawOpaqueObjects	25
▶ (RP 3:0) DrawSkybox	1
Resolve Color	
Clear (color)	
▶ (RP 4:0) CopyColor	1
Resolve Color	
Clear (color+depth+stencil)	
▼ (RP 6:0) RG_SMAAEdgeDetection	1
Draw Procedural	
Clear (color)	
▶ (RP 7:0) RG_SMAABlendWeight	1
Clear (color)	
▼ (RP 8:0) RG_SMAANeighborhoodBlend	1
Draw Procedural	
Clear (color)	
Clear (color)	
Clear (color)	
Clear (color)	
▶ Depth of Field - Bokeh	5
Clear (color)	
▶ (RP 9:0) PaniniProjection	1
▶ Bloom	16
▼ (RP 10:0) RG_UberPost	1
Draw Procedural	
▶ (RP 10:1) DrawScreenSpaceUI	1



Deferred rendering en GTAV (2013)



Render pipelines

Render pipeline

A render pipeline performs a series of operations that take the contents of a scene, and displays them on a screen.

Render pipelines

Render pipeline

A render pipeline performs a series of operations that take the contents of a scene, and displays them on a screen.

Render paths

A rendering path is the series of operations that draws and lights GameObjects the camera sees. Different rendering paths have different capabilities and performance characteristics.

Unity render pipelines

- Built-In Render Pipeline
- Universal Render Pipeline (URP)
- High Definition Render Pipeline (HDRP)

Y esto, ¿cómo se hace?



Render pipelines

Y otras cosas :)

¿Preguntas?



Facultad
de
Informática



UNIVERSIDAD COMPLUTENSE
MADRID